

An Introduction to
Generalized Algebraic Datatypes

Noé De Santo

What is a (Generalized) Algebraic Datatype?

End goal: *Generalized Algebraic Datatypes* (GADTs)

What is a (Generalized) Algebraic Datatype?

End goal: *Generalized Algebraic Datatypes* (GADTs)

- What are they?
- How do you use them?
- What are they good for?

What is a (Generalized) Algebraic Datatype?

End goal: *Generalized Algebraic Datatypes* (GADTs)

- What are they?
- How do you use them?
- What are they good for?

But before that: what even are *Algebraic Datatypes* (ADTs)?

Algebraic Datatypes

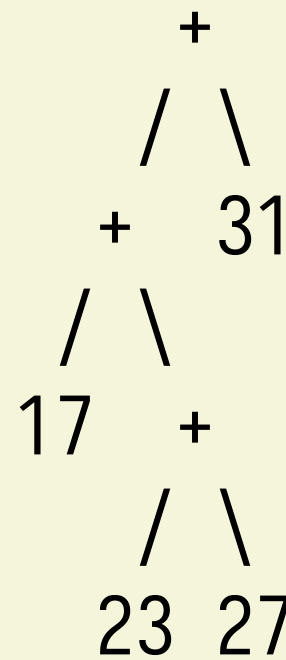
A Motivating Example: (Binary) Trees

Let's start with some (live-) coding!

A Motivating Example: (Binary) Trees

Let's start with some (live-) coding!

Our case study will be *full binary trees*:



i.e., binary trees such that each node has 0 or 2 children (but never 1).

Trees: Failures & Successes

Three versions:

Trees: Failures & Successes

Three versions:

- Full OOP (dynamic dispatch)

Trees: Failures & Successes

Three versions:

- Full OOP (dynamic dispatch)

Unsafe APIs

Trees: Failures & Successes

Three versions:

- Full OOP (dynamic dispatch)
- Pattern-matching OOP (type tests)

Unsafe APIs

Trees: Failures & Successes

Three versions:

- Full OOP (dynamic dispatch)
- Pattern-matching OOP (type tests)

Unsafe APIs

No exhaustiveness checking

Trees: Failures & Successes

Three versions:

- Full OOP (dynamic dispatch)
- Pattern-matching OOP (type tests)
- Algebraic Datatypes

Unsafe APIs

No exhaustiveness checking

Trees: Failures & Successes

Three versions:

- Full OOP (dynamic dispatch)
- Pattern-matching OOP (type tests)
- Algebraic Datatypes

Unsafe APIs

No exhaustiveness checking

Food for thoughts:

Some languages do not allow type tests

Anatomy of an ADT

```
//    ↓ Name of the defined type
enum List [T] // ← Type parameter(s)
  //    ↓ Constructor name(s)
  case Nil ()                extends List[T]
  case Cons(h: T, t: List[T]) extends List[T]
  //    ↑ Constructor(s)' payload
```

Anatomy of an ADT

```
//    ↓ Name of the defined type
enum List [T] // ← Type parameter(s)
  //    ↓ Constructor name(s)
  case Nil ()                extends List[T]
  case Cons(h: T, t: List[T]) extends List[T]
  //    ↑ Constructor(s)' payload
```

Or, in plain english:

*A List of Ts is **either**:*

- 1. a Nil, made out of "nothing"*
- 2. a Cons, made out of a T and a List[T]*

and cannot be anything else.

Anatomy of an ADT

```
//    ↓ Name of the defined type
enum List [T] // ← Type parameter(s)
//    ↓ Constructor name(s)
case Nil ()           extends List[T]
case Cons(h: T, t: List[T]) extends List[T]
//    ↑ Constructor(s)' payload
```

Recursive & Parametric!

Or, in plain english:

A List of **T**s is **either**:

1. a Nil, made out of "nothing"
2. a Cons, made out of a T and a **List[T]**

and cannot be anything else.

Anatomy of an ADT

```
//    ↓ Name of the defined type
enum List [T] // ← Type parameter(s)
//    ↓ Constructor name(s)
case Nil ()
case Cons(h: T, t: List[T])
//    ↑ Constructor(s)' payload
```

Recursive & Parametric!

extends List[T]

extends List[T]

Often optional (more on that later...)

Or, in plain english:

*A List of Ts is **either**:*

- 1. a Nil, made out of "nothing"*
- 2. a Cons, made out of a T and a List[T]*

and cannot be anything else.

Anatomy of an ADT

```
//    ↓ Name of the defined type
enum List [T] // ← Type parameter(s)
//    ↓ Constructor name(s)
case Nil ()           extends List[T]
case Cons(h: T, t: List[T]) extends List[T]
//    ↑ Constructor(s)' payload
```

Recursive & Parametric!

Often optional (more on that later...)

Or, in plain english:

*A List of Ts is **either**:*

**Finite list of constructors ⇒
patterns can be checked for exhaustiveness**

- 1. a Nil, made out of "nothing"*
- 2. a Cons, made out of a T and a List[T]*

and cannot be anything else.

Some More Examples of ADTs

```
enum Bool {  
  case True()  
  case False()  
}
```

```
enum Pair[S,T] {  
  case Pack(s: S, t: T)  
}
```

```
enum Option[T] {  
  case None()  
  case Some(t: T)  
}
```

Some More Examples of ADTs

```
enum Bool {  
  case True()  
  case False()  
}
```

```
enum Pair[S,T] {  
  case Pack(s: S, t: T)  
}
```

```
enum Option[T] {  
  case None()  
  case Some(t: T)  
}
```

```
enum Json { // Inspired by the circe library  
  case JNull  
  case JBool(b: Boolean)  
  case JNumber(n: java.math.BigDecimal)  
  case JString(s: String)  
  case JArray(a: Array[Json])  
  case JObject(m: Map[String, Json])  
}
```

Pattern Matching: So Good Everyone Wants It

Pattern matching used to be limited to functional languages, but...

Pattern Matching: So Good Everyone Wants It

Pattern matching used to be limited to functional languages, but...

- Rust:

"enums" & pattern matching

Since release (2012)

Pattern Matching: So Good Everyone Wants It

Pattern matching used to be limited to functional languages, but...

- [Rust](#):
"enums" & pattern matching **Since release (2012)**
- [Python](#):
Structural pattern matching **Since Python 3.10 (2021)**

Pattern Matching: So Good Everyone Wants It

Pattern matching used to be limited to functional languages, but...

- [Rust](#):
"enums" & pattern matching **Since release (2012)**
- [Python](#):
Structural pattern matching **Since Python 3.10 (2021)**
- [Java](#):
"sealed" class hierarcies & "switch" **Since Java21 (2023)**

Pattern Matching: So Good Everyone Wants It

Pattern matching used to be limited to functional languages, but...

- [Rust](#):
"enums" & pattern matching Since release (2012)
- [Python](#):
Structural pattern matching Since Python 3.10 (2021)
- [Java](#):
"sealed" class hierarcies & "switch" Since Java21 (2023)
- [C++](#):
"In discussion" since (at least) 2014

Pattern Matching: So Good Everyone Wants It

Pattern matching used to be limited to functional languages, but...

- [Rust](#):
"enums" & pattern matching Since release (2012)
- [Python](#):
Structural pattern matching Since Python 3.10 (2021)
- [Java](#):
"sealed" class hierarcies & "switch" Since Java21 (2023)
- [C++](#):
"In discussion" since (at least) 2014

ADTs may not be **required** for pattern matching,
but they are what makes it **type-safe**.

Type safety: things do not go wrong at runtime (i.e., no exceptions)

Pattern Matching: So Good Everyone Wants It

Pattern matching used to be limited to functional languages, but...

- [Rust](#):
"enums" & pattern matching Since release (2012)
- [Python](#):
Structural pattern matching Since Python 3.10 (2021)
- [Java](#):
"sealed" class hierarcies & "switch" 3)
- [C++](#):
"In discussion" since (at least) 2014

Even if you don't buy into ADTs:
Please give pattern matching a try

ADTs may not be **required** for pattern matching,
but they are what makes it **type-safe**.

Type safety: things do not go wrong at runtime (i.e., no exceptions)

Beyond Appearances: ADTs for Performance

We have three implementations; which one is the fastest?

Beyond Appearances: ADTs for Performance

We have three implementations; which one is the fastest?

<u>Implementation</u>	<u>Throughput (ops/s)</u>	<u>Error (ops/s)</u>
-----------------------	---------------------------	----------------------

Throughput: "How many times can the function run in a second."

Error: "How much the result varies if we repeat the experiment."

Beyond Appearances: ADTs for Performance

We have three implementations; which one is the fastest?

Implementation	Throughput (ops/s)	Error (ops/s)
OOP	123.653	1.670
Type Test	137.381	9.841
ADT	157.312	10.850

Throughput: "How many times can the function run in a second."

Error: "How much the result varies if we repeat the experiment."

A Tangent Before Moving On: What Makes Them "Algebraic"

Why not just call them "fancy enums"?

```
enum Tree[X] {  
  case Leaf(v: X) extends Tree[X]  
  case Fork(l: Tree[X], r: Tree[X]) extends Tree[X]  
}
```

A Tangent Before Moving On: What Makes Them "Algebraic"

Why not just call them "fancy enums"?

```
enum Tree[X] {  
  case Leaf(v: X)           extends Tree[X]  
  case Fork(l: Tree[X], r: Tree[X]) extends Tree[X]  
}
```

$\text{Tree}(X) = \text{Leaf}(v: X) \text{ "or" } \text{Fork}(l: \text{Tree}[X] \text{ "and" } r: \text{Tree}[X])$

A Tangent Before Moving On: What Makes Them "Algebraic"

Why not just call them "fancy enums"?

```
enum Tree[X] {  
  case Leaf(v: X)           extends Tree[X]  
  case Fork(l: Tree[X], r: Tree[X]) extends Tree[X]  
}
```

$\text{Tree}(X) = L(x: X) \text{ "or" } F(t1: \text{Tree}[X] \text{ "and" } t2: \text{Tree}[X])$

A Tangent Before Moving On: What Makes Them "Algebraic"

Why not just call them "fancy enums"?

```
enum Tree[X] {  
  case Leaf(v: X)           extends Tree[X]  
  case Fork(l: Tree[X], r: Tree[X]) extends Tree[X]  
}
```

$\text{Tree}(X) = (X) \text{ "or" } (\text{Tree}[X] \text{ "and" } \text{Tree}[X])$

A Tangent Before Moving On: What Makes Them "Algebraic"

Why not just call them "fancy enums"?

```
enum Tree[X] {  
  case Leaf(v: X) extends Tree[X]  
  case Fork(l: Tree[X], r: Tree[X]) extends Tree[X]  
}
```

$$\text{Tree}(X) = (X) + (\text{Tree}[X] * \text{Tree}[X])$$

A Tangent Before Moving On: What Makes Them "Algebraic"

Why not just call them "fancy enums"?

```
enum Tree[X] {  
  case Leaf(v: X)           extends Tree[X]  
  case Fork(l: Tree[X], r: Tree[X]) extends Tree[X]  
}
```

$$\text{Tree}(X) = (X) + (\text{Tree}[X] * \text{Tree}[X])$$

$$T(X) = X + T(X) \cdot T(X)$$

A Tangent Before Moving On: What Makes Them "Algebraic"

Why not just call them "fancy enums"?

```
enum Tree[X] {  
  case Leaf(v: X)           extends Tree[X]  
  case Fork(l: Tree[X], r: Tree[X]) extends Tree[X]  
}
```

$$\text{Tree}(X) = (X) + (\text{Tree}[X] * \text{Tree}[X])$$

$$T(X) = X + T(X) \cdot T(X)$$

Seems like algebra to me... and more specifically: a sum of products.

A Tangent Before Moving On: What Makes Them "Algebraic"

Why not just call them "fancy enums"?

```
enum Tree[X] {  
  case Leaf(v: X)           extends Tree[X]  
  case Fork(l: Tree[X], r: Tree[X]) extends Tree[X]  
}
```

$$\text{Tree}(X) = (X) + (\text{Tree}[X] * \text{Tree}[X])$$

$$T(X) = X + T(X) \cdot T(X)$$

Seems like algebra to me... and more specifically: a sum of products.

That might seem like symbol pushing, but the connection is more than syntactic.

ADTs: Summary

Algebraic Datatypes (ADTs):

- Allow to represent finite/closed hierarchies.
- Naturally support pattern matching extremely well.
- Readily available (Rust, Scala, Java, etc.).
- Fairly efficient.

An Introduction to
Generalized Algebraic Datatypes

GADTs: Generalizing by Constraining

Generalized Algebraic Datatypes: What is there to generalize?

GADTs: Generalizing by Constraining

Generalized Algebraic Datatypes: What is there to generalize?

From the [Scala documentation](#):

```
enum Box[T] {  
  case IntBox(n: Int)      extends Box[Int]  
  case BoolBox(b: Boolean) extends Box[Boolean]  
}
```

GADTs: Generalizing by Constraining

Generalized Algebraic Datatypes: What is there to generalize?

From the [Scala documentation](#):

```
enum Box[T] {  
  case IntBox(n: Int)      extends Box[Int]  
  case BoolBox(b: Boolean) extends Box[Boolean]  
}
```

I was *very* confused the first time I saw that example...

GADTs: Generalizing by Constraining

Generalized Algebraic Datatypes: What is there to generalize?

From the [Scala documentation](#):

```
enum Box[T] {  
  case IntBox(n: Int)      extends Box[Int]  
  case BoolBox(b: Boolean) extends Box[Boolean]  
}
```

I was *very* confused the first time I saw that example...

The important part is that, unlike all of our previous examples, the extended type is not `Box[T]`, but `Box` instantiated with a concrete type, either `Int` or `Boolean`.

GADTs: Generalizing by Constraining

Generalized Algebraic Datatypes: What is the
From the [Scala documentation](#):

Subsidiary question:
Why would you want to do that?

```
enum Box[T] {  
  case IntBox(n: Int)      extends Box[Int]  
  case BoolBox(b: Boolean) extends Box[Boolean]  
}
```

I was *very* confused the first time I saw that example...

The important part is that, unlike all of our previous examples, the extended type is not `Box[T]`, but `Box` instantiated with a concrete type, either `Int` or `Boolean`.

An Example to Motivate GADTs

Rather than try to explain what GADTs are outright, let's answer another question, that will naturally lead us to GADTs.

An Example to Motivate GADTs

Rather than try to explain what GADTs are outright, let's answer another question, that will naturally lead us to GADTs.

So anyway: how big is a Boolean?

An Example to Motivate GADTs

Rather than try to explain what GADTs are outright, let's answer another question, that will naturally lead us to GADTs.

So anyway: how big is a Boolean?

Empirical Answer: 8 Bits!

A Tangent: Booleans Are 8bits Big?!?

From the JVM's specification:

2.3.4. The boolean Type

Although the Java Virtual Machine defines a boolean type, it only provides very limited support for it. There are no Java Virtual Machine instructions solely dedicated to operations on boolean values. Instead, expressions in the Java programming language that operate on boolean values are compiled to use values of the Java Virtual Machine `int` data type.

The Java Virtual Machine does directly support boolean arrays. Its *newarray* instruction ([§newarray](#)) enables creation of boolean arrays. Arrays of type boolean are accessed and modified using the byte array instructions *baload* and *bastore* ([§baload](#), [§bastore](#)).

In Oracle's Java Virtual Machine implementation, boolean arrays in the Java programming language are encoded as Java Virtual Machine byte arrays, using 8 bits per boolean element.

The Java Virtual Machine encodes boolean array components using 1 to represent true and 0 to represent false. Where Java programming language boolean values are mapped by compilers to values of Java Virtual Machine type `int`, the compilers must use the same encoding.

A Tangent: Booleans Are 8bits Big?!?

From the JVM's specification:

2.3.4. The boolean Type

Although the Java Virtual Machine does not directly support boolean arrays, the Java programming language compilers that operate on boolean values are compiled to use values of the Java Virtual Machine `int` data type.

Also (somewhat) true in C:

Boolean type

`bool` — integer type, capable of holding one of the two values: `true` or `false`. The value of `sizeof(bool)` is implementation defined and might differ from `1`.

The Java Virtual Machine does not directly support boolean arrays. Its `newarray` instruction ([§newarray](#)) enables creation of boolean arrays. Arrays of type `boolean` are accessed and modified using the byte array instructions `baload` and `bastore` ([§baload](#), [§bastore](#)).

In Oracle's Java Virtual Machine implementation, `boolean` arrays in the Java programming language are encoded as Java Virtual Machine byte arrays, using 8 bits per `boolean` element.

The Java Virtual Machine encodes boolean array components using 1 to represent `true` and 0 to represent `false`. Where Java programming language boolean values are mapped by compilers to values of Java Virtual Machine type `int`, the compilers must use the same encoding.

GADTs: Generalizing by Constraining (Redux)

```
enum SmartArray[T] {  
  case Default(array: Array[T])      extends SmartArray[T]  
  case Compact(array: CompactArray) extends SmartArray[Boolean]  
}
```

tldr; GADTs allow some constructors to *restrict* the instantiation of the type parameter to a specific type.

Case in point: Compact requires SmartArray's type parameter to be Boolean.

Compact Arrays: Not a Made Up Usecase

This example was adapted from a Jane Street blog post (about OCaml).

Compact Arrays: Not a Made Up Usecase

This example was adapted from a Jane Street blog post (about OCaml).

As I said at the beginning, this is really just an example of the more general theme. GADTs [...]; they're a powerful mechanism for building easy to use abstractions that give you more precise control of your memory representation. And getting the right memory representation is often critical for building high performance applications.

A Case Study on
Generalized Algebraic Datatypes

A Refresher(?) on Red-Black Trees

Red-Black Trees are (approximately) balanced binary search trees.

They guarantee a complexity of $\log(n)$ for all basic operations:

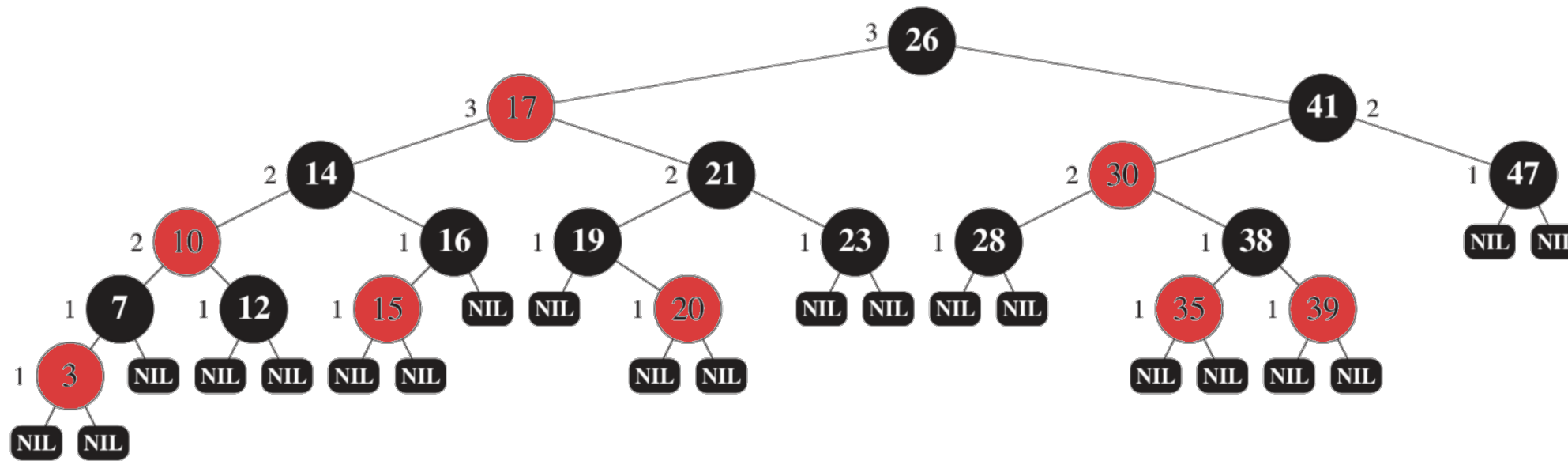
- insertion
- deletion
- membership-test

A Refresher(?) on Red-Black Trees

Red-Black Trees are (approximately) balanced binary search trees.

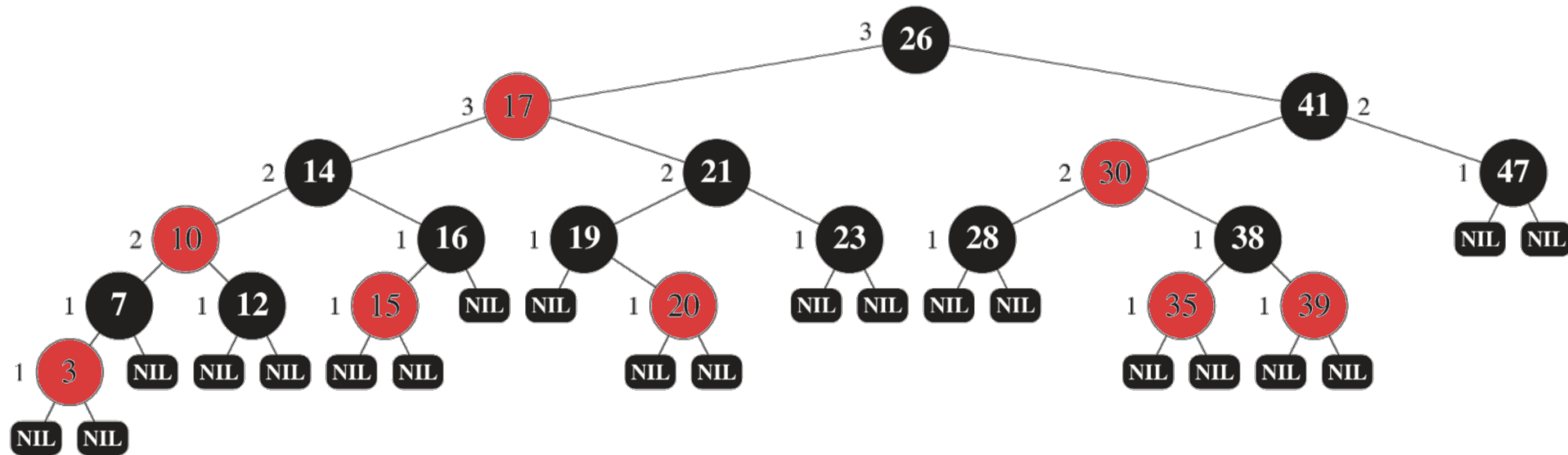
They guarantee a complexity of $\log(n)$ for all basic operations:

- insertion
- deletion
- membership-test



GADTs as Lightweight Verification

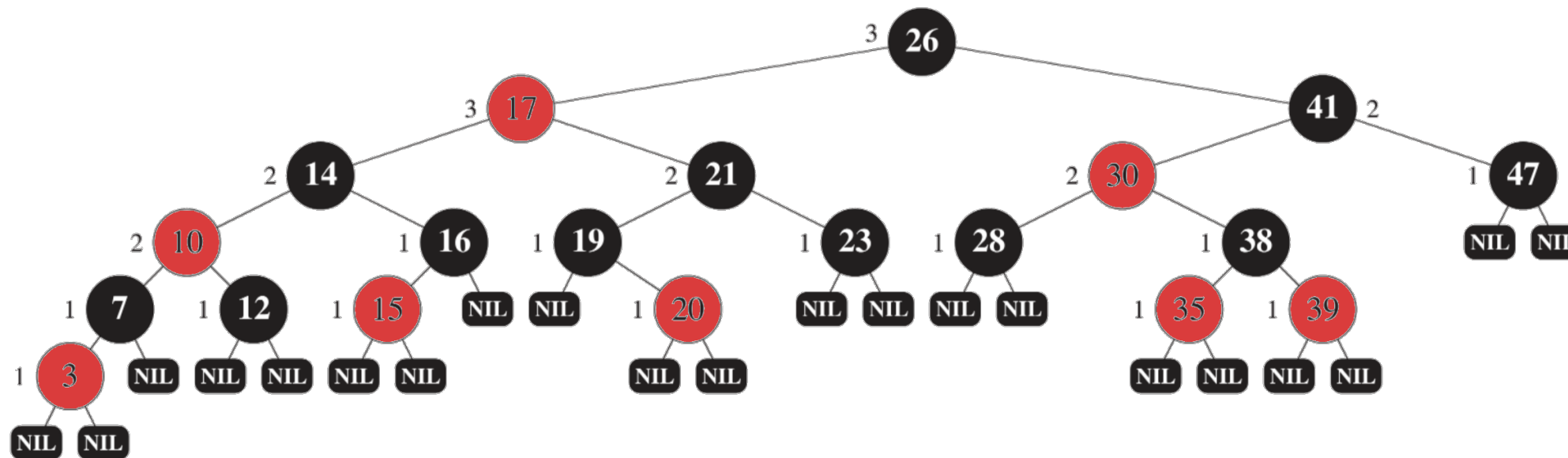
The approximate balanced part is achieved by maintaining 5 invariants:



GADTs as Lightweight Verification

The approximate balanced part is achieved by maintaining 5 invariants:

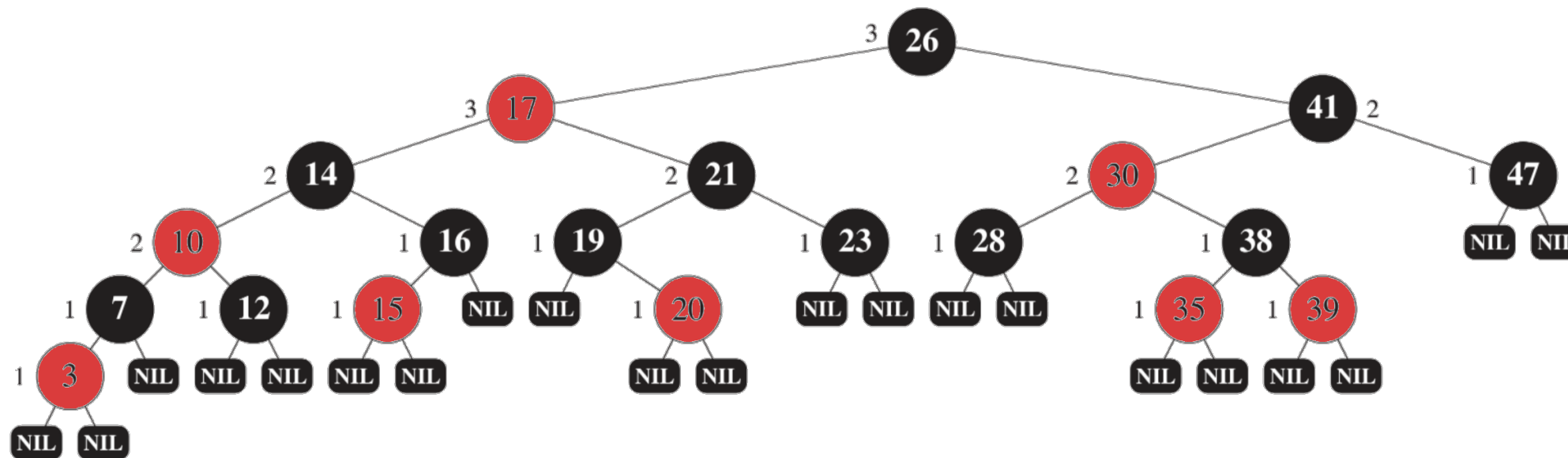
1. Every node is either **Red** or **Black**;
2. The root is **Black**;
3. Every leaf is **Black**;
4. If a node is **Red**, then both its children are **Black**;
5. For each node, all simple paths from the node to descendant leaves contain the same number of **Black** nodes.



GADTs as Lightweight Verification

The approximate balanced part is achieved by maintaining 5 invariants:

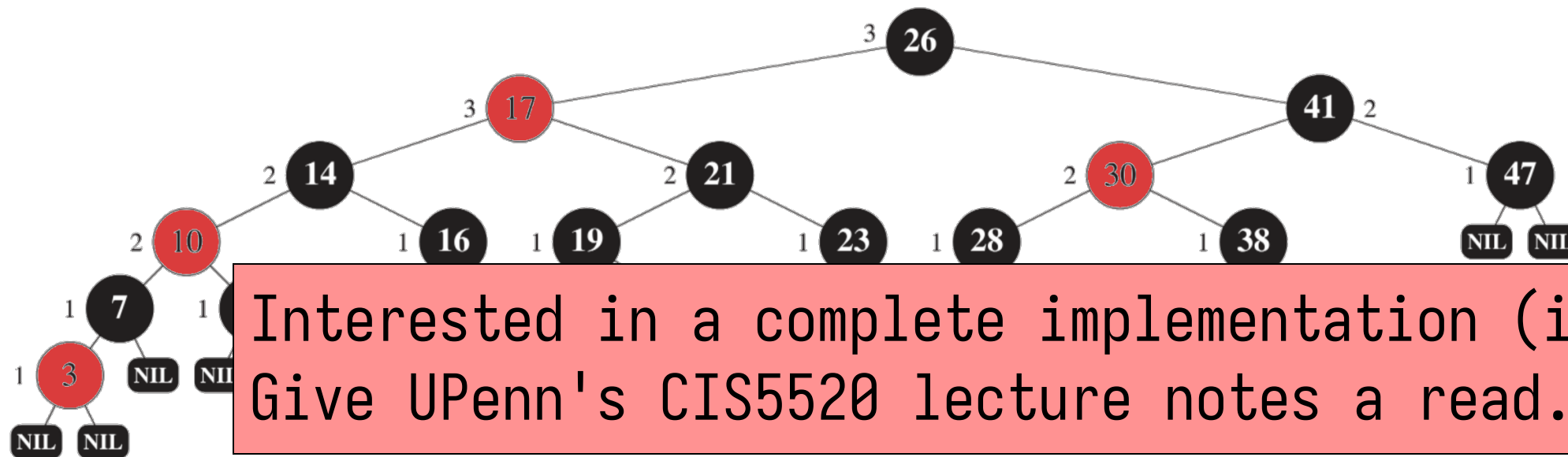
1. Every node is either **Red** or **Black**;
2. The root is **Black**;
3. Every leaf is **Black**;
4. If a node is **Red**, then both its children are **Black**;
5. For each node, all simple paths from the node to descendant leaves contain the same number of **Black** nodes.



GADTs as Lightweight Verification

The approximate balanced part is achieved by maintaining 5 invariants:

1. Every node is either **Red** or **Black**;
2. The root is **Black**;
3. Every leaf is **Black**;
4. If a node is **Red**, then both its children are **Black**;
5. For each node, all simple paths from the node to descendant leaves contain the same number of **Black** nodes.



A Scala Oddity: ADTs Declare Many Types

A Scala Oddity: ADTs Declare Many Types

```
enum Color { // Declares 3 types: Color, Red, Black
  case Red()
  case Black()
}
```

```
val _: Color = Red()
```

```
val _: Red = Red()
```

```
val _: Black = Red() // Type error
```

```
val _ = Red() // Type defaults to Color
```

A Scala Oddity: ADTs Declare Many Types

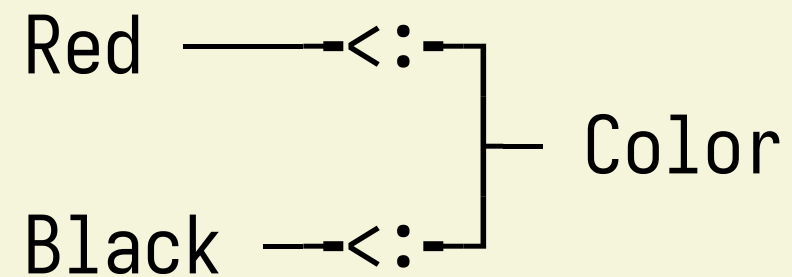
```
enum Color { // Declares 3 types: Color, Red, Black
  case Red()
  case Black()
}
```

```
val _: Color = Red()
```

```
val _: Red = Red()
```

```
val _: Black = Red() // Type error
```

```
val _ = Red() // Type defaults to Color
```



$\rightarrow$: is the Scala symbol for "subtype"

GADTs: Summary

Generalized Algebraic Datatypes (GADTs):

- Generalize ADTs by allowing constructors to restrict the instantiation of type parameters.
- Seen two use cases:
 - Control over data representation (Compact Boolean Arrays).
 - Lightweight verification (Red-Black Tree).

Backup

Null References: The Billion Dollar Mistake

I call it my billion-dollar mistake. It was the invention of the null reference in 1965. [...]

Tony Hoare, "inventor" of the null references/pointers

Null References: The Billion Dollar Mistake

I call it my billion-dollar mistake. It was the invention of the null reference in 1965. [...]

Tony Hoare, "inventor" of the null references/pointers

Interestingly, they also mention:

The types of the pointer could then be implemented as a union of either a pointer to the null type, or a pointer to the type.

Null References: The Billion Dollar Mistake

I call it my billion-dollar mistake. It was the invention of the null reference in 1965. [...]

Tony Hoare, "inventor" of the null references/pointers

Interestingly, they also mention:

The types of the pointer could then be implemented as a union of either a pointer to the null type, or a pointer to the type.

```
enum Option[T] {  
    case None(/*v: Null*/)  
    case Some(v: T)  
}
```